Leaf and Non-Leaf Node Counting in a General Tree

Output Analysis

1. Analysis of the Program Output

The sample tree loaded by menu option 7 has 12 nodes (1 to 12). The program produced the following results, which are consistent with each other:

- **Tree structure (option 3):** node 1 is the root with children 2, 3, 4. Node 2 has children 5 and 6; node 3 has child 7; node 4 has children 8, 9, 10; node 6 has children 11 and 12.
- **Leaf nodes:** 5, 7, 8, 9, 10, 11, 12 (7 leaves). **Non-leaf nodes:** 1, 2, 3, 4, 6 (5 non-leaves). Check: 7 + 5 = 12 = total nodes.
- **Nodes at each level (option 5):** Level 0 has 1 node, Level 1 has 3 nodes (2 3 4), Level 2 has 6 nodes (5 6 7 8 9 10) and Level 3 has 2 nodes (11 12). Sum: 1 + 3 + 6 + 2 = 12. Height is 4 levels.
- **Child query (option 6):** for example, the children of node 6 are 11 and 12; the children of node 4 are 8, 9 and 10; a leaf such as node 5 returns no children.

Every leaf in the displayed tree is tagged *(leaf)*, and every untagged node has at least one child, so the display, the leaf/non-leaf list and the level counts all agree.

2. Experiments with Several Input Sizes

The counting, level-counting and child-query operations were repeated for n = 10, 100, 1,000, 10,000 and 100,000 nodes on three tree shapes: a **random tree** (each new node attached to a random existing node, average case), a **chain** (every node has one child, maximum height) and a **star** (all nodes are children of the root, maximum width). Times are the average of repeated runs, measured on a prototype of the same algorithms; absolute values depend on the machine, so the growth trend matters more than the exact number. The child query searches for the last inserted node, then lists its children.

Table 1: Random tree (average case)

n	Leaf nodes	Non-leaf nodes	Levels (height)	Max nodes in a level	Node visits (count)	Count time (ms)	Level-count time (ms)	Child query visits
10	6	4	4	6	10	0.001	0.002	7
100	46	54	11	25	100	0.008	0.038	14
1,000	498	502	18	167	1,000	0.112	0.182	310
10,000	4,962	5,038	23	1,290	10,000	1.246	2.226	979
100,000	49,800	50,200	25	12,982	100,000	40.22	61.13	27,047

Table 2: Chain tree (deepest case, height = n)

n	Leaf nodes	Non-leaf nodes	Levels (height)	Max nodes in a level	Node visits (count)	Count time (ms)	Level-count time (ms)	Child query visits
10	1	9	10	1	10	0.001	0.002	10
100	1	99	100	1	100	0.008	0.018	100
1,000	1	999	1000	1	1,000	0.089	0.252	1,000
10,000	1	9,999	10000	1	10,000	1.053	2.230	10,000
100,000	1	99,999	100000	1	100,000	10.88	21.57	100,000

Table 3: Star tree (widest case, height = 2)

n	Leaf nodes	Non-leaf nodes	Levels (height)	Max nodes in a level	Node visits (count)	Count time (ms)	Level-count time (ms)	Child query visits
10	9	1	2	9	10	0.001	0.002	2
100	99	1	2	99	100	0.008	0.016	2
1,000	999	1	2	999	1,000	0.079	0.176	2
10,000	9,999	1	2	9,999	10,000	0.911	2.259	2
100,000	99,999	1	2	99,999	100,000	9.662	24.45	2

3. Observations from the Experiments

- **Node visits equal n exactly** for leaf/non-leaf counting and for level counting in every shape (10, 100, 1,000, 10,000, 100,000). Each node is visited once, regardless of tree shape.
- **Time grows linearly with n.** For the chain and star trees the time rises about 10 times for every 10 times increase in n (for example, chain counting: 0.088 ms at 1,000 and 1.05 ms at 10,000). The random tree follows the same pattern up to 10,000 nodes; its 100,000 run is slower than a pure linear trend (40 ms instead of about 12 ms), which is most likely a memory/cache effect of scattered nodes, not extra work, since the visit count is still exactly n.
- **Level counting is about 2 times slower than leaf counting** because it uses a queue and stores a depth value with every node, but both are O(n).
- **Leaf proportion:** in the random tree about half of the nodes are leaves (49.8% at n = 100,000). In the chain tree there is only 1 leaf, and in the star tree n - 1 leaves.
- **Height depends on shape, not only on n:** 25 levels for a random tree of 100,000 nodes (roughly logarithmic), 100,000 for the chain and 2 for the star.
- **Child query cost depends on how the node is found.** It visited only 2 nodes in the star tree, about 27,000 in the random tree, and all n nodes in the chain. Once the node is located, listing its children costs only O(d), where d is the number of children.

4. Time Complexity

- Displaying the tree: O(n), each node printed once.
- Counting leaf and non-leaf nodes and displaying their values: O(n) to traverse the tree; printing the values adds O(n), so the total is still O(n).
- Counting nodes at each level: O(n) with a level-order (BFS) or depth-tracking traversal.
- Height calculation: O(n).
- Query children of a selected node: O(n) in the worst case to find the node (O(1) if a node-to-pointer map or array is kept), plus O(d) to print its d children.
- Adding a child: O(n) to locate the parent by search, plus O(1) to attach the child (O(1) overall with a lookup table).
- Loading the sample tree: O(n) insertions with the search cost above; for the fixed 12-node sample it is constant work.

5. Space Complexity

- Tree storage: O(n) for n nodes and n - 1 parent-to-child links.
- Leaf and non-leaf lists: O(n) in total, because each node goes into exactly one of the two lists.
- Level counts: O(h) for the counters, plus the queue in BFS, which holds at most one level, up to O(w) where w is the maximum width (w = n - 1 in the star tree).

- Recursive traversal call stack: O(h), which is O(n) for a chain, so very deep trees could cause stack overflow in a recursive implementation.
- Overall auxiliary space: O(n) in the worst case (O(h) for the stack-based DFS on a balanced tree).

Here, n is the number of nodes, h is the height of the tree, d is the number of children of the selected node, and w is the maximum number of nodes in one level.

6. Conclusion

The program's output is correct for the sample tree (7 leaves, 5 non-leaves, 4 levels with 1, 3, 6 and 2 nodes). Repeating the experiments over input sizes from 10 to 100,000 shows that leaf and non-leaf counting and level counting visit each node exactly once and scale linearly, O(n), for any tree shape. Tree shape affects only the height, the width, memory use of the traversal (stack for deep trees, queue for wide trees) and the cost of locating a node for the child query. The three suggested modifications (showing the actual leaf and non-leaf values, counting nodes per level, and querying the children of a node) add no more than O(n) extra time and O(n) extra space.